Memoria Práctica 2

Carlos Méndez Barrasa y Martín Darío Marmolejo Diaz, grupo: 1312 equipo: 7

1. Introducción

En esta práctica hemos implementado un videoclub de películas en el que los usuarios se registran, consultan el catálogo, recargan su saldo, compran películas mediante un carrito de la compra y las valoran. A diferencia de la práctica anterior, los datos ya no se guardan en ficheros sino en una base de datos PostgreSQL, y hemos llevado buena parte de la lógica de negocio a la propia base de datos mediante triggers y procedimientos almacenados.

El sistema sigue el enfoque de microservicios de la práctica 1: `user.py` (gestión de usuarios, puerto 5050) y `api.py` (videoclub, puerto 5051) se ejecutan en contenedores independientes y acceden a la misma base de datos, que se despliega en un tercer contenedor. Toda la funcionalidad se comprueba con el cliente `client.py`, y en `optimizacion.sql` estudiamos con EXPLAIN cómo mejorar la consulta de ventas.

2. Diseño de la base de datos

El diseño se recoge en el siguiente diagrama Entidad-Relación. Las claves primarias aparecen subrayadas y las relaciones N:M con atributos propios llevan esos atributos en elipses.

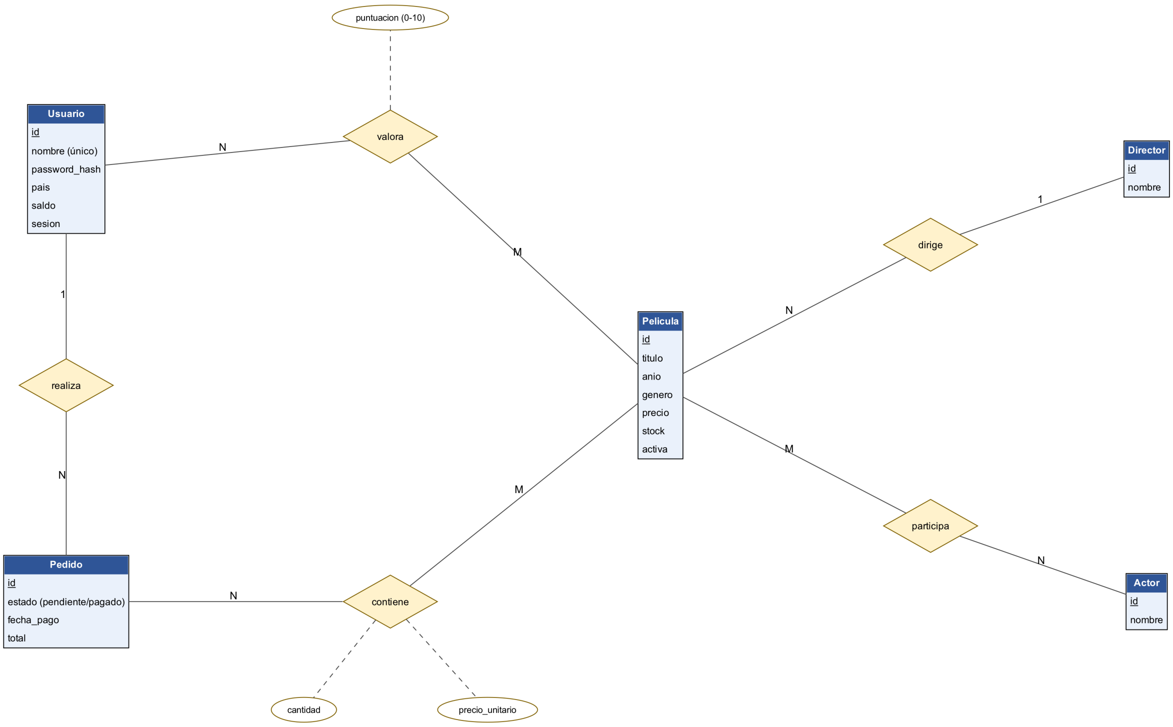


Figura 1. Diagrama E-R de la base de datos

A partir del diagrama se obtienen ocho tablas (`schema.sql`):

- **Usuario**: nombre (único), hash de la contraseña, país, saldo y un contador `sesion` que usamos para el logout (apartado 4).
- **Director** y **Actor**: solo id y nombre. Cada película tiene un director (clave foránea `director_id` en Pelicula) y varios actores a través de la tabla **PeliculaActor**.
- **Pelicula**: título, año, género, precio, stock y el campo `activa`.
- **Pedido**: representa tanto el carrito como las compras. Un pedido está en estado `pendiente` (es el carrito del usuario) o `pagado` (es una venta) y guarda la fecha de pago y el total.

- **PedidoPelicula**: las películas de cada pedido, con la cantidad y el precio unitario.
- **Valoracion**: la puntuación de un usuario a una película. Su clave primaria es (usuario, película), así que cada usuario solo puede tener un voto por película.

Decisiones de diseño

Restricciones en la propia base de datos. Hemos puesto CHECK para que el saldo, el precio y el stock no sean negativos, que la cantidad sea mayor que 0, que la puntuación esté entre 0 y 10, y que un pedido pagado tenga siempre fecha de pago y uno pendiente no. Así ninguna operación, venga de donde venga, puede dejar datos incoherentes, y la API puede limitarse a traducir los errores a códigos HTTP.

Un único carrito por usuario. El carrito es simplemente el pedido pendiente del usuario. Para garantizar que no pueda haber dos, hemos creado un índice único parcial: `CREATE UNIQUE INDEX ... ON Pedido (usuario_id) WHERE estado = 'pendiente'`. La restricción solo se aplica a los pedidos pendientes, de modo que un usuario puede tener tantos pedidos pagados como quiera.

Precio congelado al añadir al carrito. Cada línea de PedidoPelicula guarda el precio que tenía la película en el momento de añadirla. Si el precio sube después, el cliente paga el precio que aceptó y el historial de ventas no cambia.

Borrado lógico de películas. En un videoclub las películas vendidas forman parte del historial, por lo que borrar una película no elimina la fila sino que pone `activa = false`. La película desaparece del catálogo y no se puede comprar ni votar, pero sus ventas y valoraciones se conservan.

Borrado de usuarios. Las claves foráneas hacia Usuario y Pedido tienen ON DELETE CASCADE, así que al borrar un usuario se borran sus pedidos, sus líneas y sus valoraciones. Antes de borrarlo se vacía su carrito para devolver las unidades al stock (apartado 3).

Stock. Las unidades se descuentan al añadir la película al carrito, no al pagar, tal y como plantea el enunciado. Así un usuario nunca puede tener en el carrito más unidades de las que existen.

La base de datos se crea automáticamente: el contenedor de PostgreSQL ejecuta `schema.sql` y `populate.sql` la primera vez que arranca y guarda los datos en el volumen `postgres_data`. `populate.sql` inserta 10 usuarios de 4 países, 10 películas, 7 directores, 10 actores, valoraciones y 8 pedidos pagados de 2025 y 2026. Los pedidos se crean con el mismo procedimiento que usa la API, de modo que el stock, los totales y los saldos los calculan los triggers.

3. Triggers y procedimientos almacenados

Triggers

actualizar_carrito se ejecuta después de cada INSERT, UPDATE o DELETE en PedidoPelicula, es decir, cada vez que cambia el contenido de un carrito. Primero comprueba que el pedido sigue pendiente (no se puede modificar un pedido pagado) bloqueando su fila con `FOR UPDATE` para evitar condiciones de carrera entre peticiones simultáneas. Después ajusta el stock: al añadir resta las unidades y lanza 'Stock insuficiente' si no hay suficientes, al cambiar la cantidad ajusta la diferencia y al quitar una película devuelve las unidades. Por último recalcula el total del pedido como la suma de cantidad × precio_unitario.

actualizar_saldo se ejecuta después de cada UPDATE en Pedido. Impide modificar un pedido que ya está pagado y, cuando un pedido pasa de pendiente a pagado, comprueba que no esté vacío y descuenta el total del saldo del usuario, lanzando 'Saldo insuficiente' si no le llega. Como el CHECK de la tabla obliga a que el saldo no sea negativo, nunca puede quedar un pago a medias.

Cuando un trigger lanza una excepción (RAISE EXCEPTION), PostgreSQL deshace toda la operación. La API captura el error y, según el mensaje, devuelve el código adecuado (409 para falta de stock o de saldo, 400 para carrito vacío).

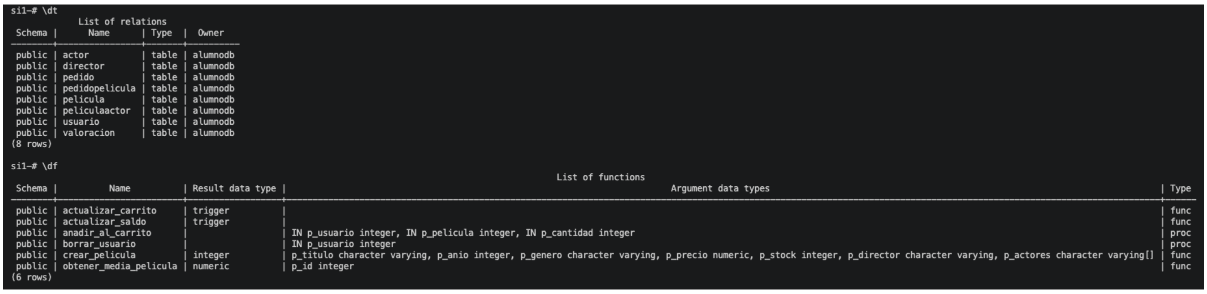
Procedimientos almacenados

Hemos implementado como procedimientos las operaciones que necesitan varias sentencias, de forma que la API las ejecuta con un único execute() como exige el enunciado:

- obtener_media_pelicula(id): devuelve la media de las puntuaciones de una película redondeada a dos decimales, o NULL si no tiene votos. Es el ejemplo que propone el enunciado y la API lo usa al consultar una película.
- crear_pelicula(...): recibe los datos de la película, el nombre del director y una lista con los nombres de los actores. Busca el director y cada actor por nombre, los crea si no existen, inserta la película, la relaciona con sus actores y devuelve su id. Si algo falla se deshace todo.
- anadir_al_carrito(usuario, película, cantidad): comprueba que la película existe y está activa, busca el carrito del usuario o lo crea si no tiene, e inserta la línea con el precio actual. Si la película ya estaba en el carrito suma las unidades (INSERT ... ON CONFLICT DO UPDATE). El trigger actualizar_carrito se encarga del stock y del total.
- borrar_usuario(usuario): vacía el carrito del usuario (el trigger devuelve las unidades al stock) y borra el usuario; sus pedidos y valoraciones se eliminan en cascada. Las unidades de los pedidos ya pagados no vuelven al stock porque se vendieron.

El pago del carrito no necesita procedimiento: es un único UPDATE que pone el pedido como pagado con la fecha actual (NOW()), y el trigger actualizar_saldo hace el resto. Tampoco hemos puesto en un procedimiento la consulta de ventas, para poder estudiar su plan de ejecución con EXPLAIN (apartado 8).

En la siguiente captura se muestran las tablas listadas en la base de datos y las funciones y procedimientos almacenados que hemos implementado:



```
si1-# \dt
             List of relations
 Schema |      Name      | Type  |  Owner
--------+----------------+-------+----------
 public | actor          | table | alumnodb
 public | director       | table | alumnodb
 public | pedido         | table | alumnodb
 public | pedidopelicula | table | alumnodb
 public | pelicula       | table | alumnodb
 public | peliculaactor  | table | alumnodb
 public | usuario        | table | alumnodb
 public | valoracion     | table | alumnodb
(8 rows)

si1-# \df
                                                                                          List of functions
 Schema |          Name          | Result data type |                                                                          Argument data types                                                                          | Type
--------+------------------------+------------------+----------------------------------------------------------------------------------------------------------------------------------------------------------------------------+------
 public | actualizar_carrito     | trigger          |                                                                                                                                                                            | func
 public | actualizar_saldo       | trigger          |                                                                                                                                                                            | func
 public | anadir_al_carrito      |                  | IN p_usuario integer, IN p_pelicula integer, IN p_cantidad integer                                                                                                         | proc
 public | borrar_usuario         |                  | IN p_usuario integer                                                                                                                                                       | proc
 public | crear_pelicula         | integer          | p_titulo character varying, p_anio integer, p_genero character varying, p_precio numeric, p_stock integer, p_director character varying, p_actores character varying[] | func
 public | obtener_media_pelicula | numeric          | p_id integer                                                                                                                                                               | func
(6 rows)
```

Figura 2. Tablas (\dt) y funciones, triggers y procedimientos (\df) de la base de datos

4. Servicio de usuarios (user.py)

Expone los siguientes endpoints en el puerto 5050. Los que modifican al usuario exigen el token en la cabecera `Authorization: Bearer <token>` y devuelven 401 si falta o no es válido.

Método y ruta	Descripción	Códigos
PUT /user	Crear usuario con nombre, contraseña y país. Devuelve uid y token	201, 400, 409 (ya existe)

Método y ruta	**Descripción**	**Códigos**
POST /user	Login con nombre y contraseña. Devuelve uid y token	200, 400, 401
POST /user/logout	Cerrar sesión: el token deja de ser válido	200, 401
PATCH /user	Cambiar la contraseña	200, 400, 401
PATCH /user/saldo	Recargar saldo con una cantidad positiva. Devuelve el saldo	200, 400, 401
DELETE /user	Borrar el propio usuario (procedimiento borrar_usuario)	200, 401

La contraseña se guarda como hash SHA-256, como en la práctica anterior. Para detectar un nombre repetido no hacemos un SELECT previo: dejamos que la restricción UNIQUE rechace el INSERT y devolvemos 409 al capturar el IntegrityError.

Token y logout. El enunciado pide que la sesión sea stateless y que el token no se guarde en la base de datos, pero también que exista logout. Con el token de la práctica 1, uuid5(secreto, uid), el token es siempre el mismo y no se puede invalidar. Nuestra solución es que el token tenga la forma `<uid>.<uuid5(secreto, uid.sesion)>`, donde `sesion` es un contador guardado en Usuario. El uid permite saber a qué usuario pertenece el token sin recorrer todos los usuarios, y la segunda parte no se puede fabricar sin el secreto. Para validar un token se lee la sesión del usuario y se recalcula. Al hacer logout se incrementa el contador, de modo que el token anterior deja de coincidir y el siguiente login genera uno nuevo. En la base de datos solo se guarda el contador, nunca el token. El secreto se genera la primera vez que arranca user.py en `shared/secret_uuid.txt`, y lo lee también api.py, igual que en la práctica 1.

5. API principal (api.py)

Todo el acceso a la base de datos se hace con SQLAlchemy Core en modo asíncrono. Al arrancar se crea un engine con `create_async_engine` y el driver asyncpg, y en cada endpoint se abre una conexión con `async with engine.begin()` (o `engine.connect()` si solo se lee), que hace commit al terminar o rollback si hay un error. Las sentencias se escriben a mano con `text()` y los valores se pasan en un diccionario de parámetros (`:nombre`), es decir, como prepared statements, de forma que los datos nunca se concatenan al SQL y no es posible la inyección SQL. Cada operación se resuelve con un único execute(): cuando hace falta información de varias tablas usamos JOIN, subconsultas o los procedimientos del apartado 3, nunca varias consultas combinadas en Python.

Método y ruta	**Descripción**	**Token**	**Códigos**
GET /peliculas	Catálogo, con filtros opcionales ?genero=, ?director=, ?actor=	No	200
GET /peliculas/<id>	Detalle con director, actores y puntuación media	No	200, 404
POST /peliculas	Crear película (procedimiento crear_pelicula)	Sí	201, 400, 401
DELETE /peliculas/<id>	Borrado lógico de la película	Sí	200, 401, 404
PUT /peliculas/<id>/valoracion	Votar de 0 a 10; si ya había votado se cambia el voto	Sí	200, 400, 401, 404

Método y ruta	**Descripción**	**Token**	**Códigos**
GET /carrito	Ver el carrito con sus películas y el total	Sí	200, 401
POST /carrito	Añadir película (procedimiento anadir_al_carrito)	Sí	200, 400, 401, 404, 409
DELETE /carrito/<id>	Quitar una película del carrito	Sí	200, 401, 404
POST /carrito/pagar	Pagar el carrito y guardar la fecha de pago	Sí	200, 400, 401, 409
GET /ventas/<año>/<país>	Pedidos pagados en un año por usuarios de un país	No	200

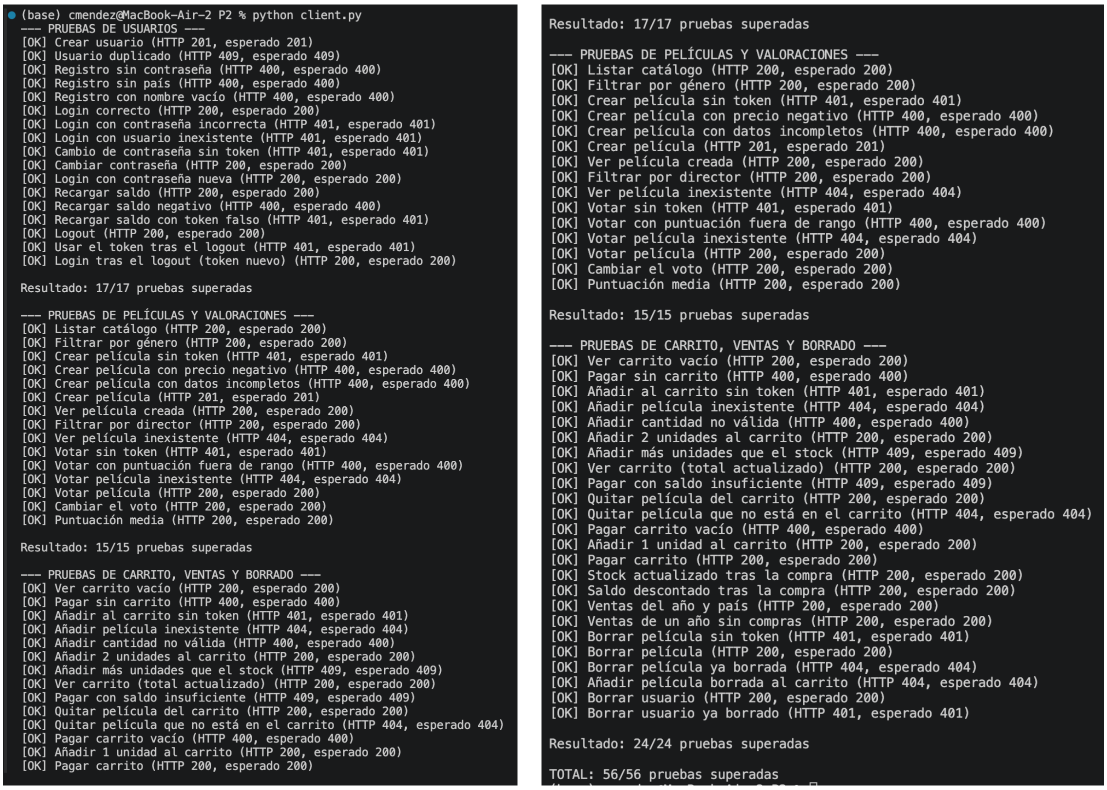
El usuario se obtiene siempre del token y nunca de la URL, así que nadie puede operar sobre el carrito de otro. Algunas decisiones que cabe destacar:

- Listado: los filtros son opcionales; si un filtro no llega se pasa NULL y esa condición no filtra nada. Usamos ILIKE con comodines para que la búsqueda no distinga mayúsculas y encuentre coincidencias parciales (por ejemplo, `?director=nolan`).
- Detalle: en una sola consulta se obtienen los datos de la película, su director (JOIN), la media (llamando a obtener_media_pelicula) y la lista de actores (ARRAY(subconsulta)).
- Crear película: la validación la hace la base de datos (NOT NULL, CHECK y tipos de las columnas); cualquier error se devuelve como 400.
- Votar: se usa INSERT ... SELECT desde Pelicula filtrando por película activa, de modo que si no existe no se inserta nada y se devuelve 404, y ON CONFLICT DO UPDATE para cambiar el voto si ya existía.
- Errores de la base de datos: los RAISE EXCEPTION de triggers y procedimientos llegan a Python como DBAPIError. Según el mensaje devolvemos 409 (stock o saldo insuficiente: la petición es correcta pero choca con el estado actual), 404 (película no disponible) o 400 (datos no válidos).

6. Cliente de pruebas (client.py)

El cliente mantiene la estructura de la práctica 1: las funciones peticion (usuarios) y peticion_api (videoclub) añaden la cabecera Authorization cuando se les pasa un token, y comprobar compara el código HTTP con el esperado y, cuando hace falta, también el contenido de la respuesta. Las 56 pruebas se dividen en tres bloques: usuarios (17), películas y valoraciones (15) y carrito, ventas y borrado (24). Cubren tanto los casos correctos como los errores: falta de token, token falso o caducado tras el logout, datos no válidos, recursos inexistentes, falta de stock y saldo insuficiente.

Varias pruebas comprueban que los triggers funcionan: el usuario de prueba recarga 50 € y crea una película de 30 € con 3 unidades; añade 2 (el total pasa a 60), intenta añadir 5 más (409, solo queda 1), intenta pagar (409, saldo insuficiente), quita la película del carrito, añade 1 y paga. Después se comprueba que el stock ha quedado en 2 y el saldo en 20. Como el nombre y el país del usuario se generan al azar en cada ejecución, la consulta /ventas de ese país y año debe devolver exactamente esa compra, y el cliente se puede ejecutar todas las veces que se quiera. Al final se borran la película y el usuario.



```
(base) cmendez@MacBook-Air-2 P2 % python client.py
--- PRUEBAS DE USUARIOS ---
[OK] Crear usuario (HTTP 201, esperado 201)
[OK] Usuario duplicado (HTTP 409, esperado 409)
[OK] Registro sin contraseña (HTTP 400, esperado 400)
[OK] Registro sin país (HTTP 400, esperado 400)
[OK] Registro con nombre vacío (HTTP 400, esperado 400)
[OK] Login correcto (HTTP 200, esperado 200)
[OK] Login con contraseña incorrecta (HTTP 401, esperado 401)
[OK] Login con usuario inexistente (HTTP 401, esperado 401)
[OK] Cambio de contraseña sin token (HTTP 401, esperado 401)
[OK] Cambiar contraseña (HTTP 200, esperado 200)
[OK] Login con contraseña nueva (HTTP 200, esperado 200)
[OK] Recargar saldo (HTTP 200, esperado 200)
[OK] Recargar saldo negativo (HTTP 400, esperado 400)
[OK] Recargar saldo con token falso (HTTP 401, esperado 401)
[OK] Logout (HTTP 200, esperado 200)
[OK] Usar el token tras el logout (HTTP 401, esperado 401)
[OK] Login tras el logout (token nuevo) (HTTP 200, esperado 200)

Resultado: 17/17 pruebas superadas

--- PRUEBAS DE PELÍCULAS Y VALORACIONES ---
[OK] Listar catálogo (HTTP 200, esperado 200)
[OK] Filtrar por género (HTTP 200, esperado 200)
[OK] Crear película sin token (HTTP 401, esperado 401)
[OK] Crear película con precio negativo (HTTP 400, esperado 400)
[OK] Crear película con datos incompletos (HTTP 400, esperado 400)
[OK] Crear película (HTTP 201, esperado 201)
[OK] Ver película creada (HTTP 200, esperado 200)
[OK] Filtrar por director (HTTP 200, esperado 200)
[OK] Ver película inexistente (HTTP 404, esperado 404)
[OK] Votar sin token (HTTP 401, esperado 401)
[OK] Votar con puntuación fuera de rango (HTTP 400, esperado 400)
[OK] Votar película inexistente (HTTP 404, esperado 404)
[OK] Votar película (HTTP 200, esperado 200)
[OK] Cambiar el voto (HTTP 200, esperado 200)
[OK] Puntuación media (HTTP 200, esperado 200)

Resultado: 15/15 pruebas superadas

--- PRUEBAS DE CARRITO, VENTAS Y BORRADO ---
[OK] Ver carrito vacío (HTTP 200, esperado 200)
[OK] Pagar sin carrito (HTTP 400, esperado 400)
[OK] Añadir al carrito sin token (HTTP 401, esperado 401)
[OK] Añadir película inexistente (HTTP 404, esperado 404)
[OK] Añadir cantidad no válida (HTTP 400, esperado 400)
[OK] Añadir 2 unidades al carrito (HTTP 200, esperado 200)
[OK] Añadir más unidades que el stock (HTTP 409, esperado 409)
[OK] Ver carrito (total actualizado) (HTTP 200, esperado 200)
[OK] Pagar con saldo insuficiente (HTTP 409, esperado 409)
[OK] Quitar película del carrito (HTTP 200, esperado 200)
[OK] Quitar película que no está en el carrito (HTTP 404, esperado 404)
[OK] Pagar carrito vacío (HTTP 400, esperado 400)
[OK] Añadir 1 unidad al carrito (HTTP 200, esperado 200)
[OK] Pagar carrito (HTTP 200, esperado 200)
```

```
Resultado: 17/17 pruebas superadas

--- PRUEBAS DE PELÍCULAS Y VALORACIONES ---
[OK] Listar catálogo (HTTP 200, esperado 200)
[OK] Filtrar por género (HTTP 200, esperado 200)
[OK] Crear película sin token (HTTP 401, esperado 401)
[OK] Crear película con precio negativo (HTTP 400, esperado 400)
[OK] Crear película con datos incompletos (HTTP 400, esperado 400)
[OK] Crear película (HTTP 201, esperado 201)
[OK] Ver película creada (HTTP 200, esperado 200)
[OK] Filtrar por director (HTTP 200, esperado 200)
[OK] Ver película inexistente (HTTP 404, esperado 404)
[OK] Votar sin token (HTTP 401, esperado 401)
[OK] Votar con puntuación fuera de rango (HTTP 400, esperado 400)
[OK] Votar película inexistente (HTTP 404, esperado 404)
[OK] Votar película (HTTP 200, esperado 200)
[OK] Cambiar el voto (HTTP 200, esperado 200)
[OK] Puntuación media (HTTP 200, esperado 200)

Resultado: 15/15 pruebas superadas

--- PRUEBAS DE CARRITO, VENTAS Y BORRADO ---
[OK] Ver carrito vacío (HTTP 200, esperado 200)
[OK] Pagar sin carrito (HTTP 400, esperado 400)
[OK] Añadir al carrito sin token (HTTP 401, esperado 401)
[OK] Añadir película inexistente (HTTP 404, esperado 404)
[OK] Añadir cantidad no válida (HTTP 400, esperado 400)
[OK] Añadir 2 unidades al carrito (HTTP 200, esperado 200)
[OK] Añadir más unidades que el stock (HTTP 409, esperado 409)
[OK] Ver carrito (total actualizado) (HTTP 200, esperado 200)
[OK] Pagar con saldo insuficiente (HTTP 409, esperado 409)
[OK] Quitar película del carrito (HTTP 200, esperado 200)
[OK] Quitar película que no está en el carrito (HTTP 404, esperado 404)
[OK] Pagar carrito vacío (HTTP 400, esperado 400)
[OK] Añadir 1 unidad al carrito (HTTP 200, esperado 200)
[OK] Pagar carrito (HTTP 200, esperado 200)
[OK] Stock actualizado tras la compra (HTTP 200, esperado 200)
[OK] Saldo descontado tras la compra (HTTP 200, esperado 200)
[OK] Ventas del año y país (HTTP 200, esperado 200)
[OK] Ventas de un año sin compras (HTTP 200, esperado 200)
[OK] Borrar película sin token (HTTP 401, esperado 401)
[OK] Borrar película (HTTP 200, esperado 200)
[OK] Borrar película ya borrada (HTTP 404, esperado 404)
[OK] Añadir película borrada al carrito (HTTP 404, esperado 404)
[OK] Borrar usuario (HTTP 200, esperado 200)
[OK] Borrar usuario ya borrado (HTTP 401, esperado 401)

Resultado: 24/24 pruebas superadas

TOTAL: 56/56 pruebas superadas
```

Figura 3. Salida de client.py con los servicios levantados con Docker

7. Docker

Mantenemos la configuración de la práctica 1: una única imagen (python:3.13.9) que instala requirements.txt, ahora con SQLAlchemy y asyncpg, y que usan los dos microservicios, cada uno con su comando y su puerto. Los contenedores montan el directorio del proyecto como volumen y conservamos la opción network: host en la construcción para que funcione en los ordenadores del laboratorio.

Hemos añadido el servicio db con la imagen oficial postgres:17, que crea el usuario alumnodb con contraseña 1234 y la base de datos si1. schema.sql y populate.sql se montan en /docker-entrypoint-initdb.d, por lo que se ejecutan automáticamente la primera vez que se levanta el contenedor, y los datos persisten en el volumen postgres_data. Los servicios esperan a que la base de datos esté lista gracias a un healthcheck con pg_isready, y reciben la URL de conexión por la variable de entorno DATABASE_URL, en la que el host es db, el nombre del servicio dentro de la red de Docker.

Con las versiones más recientes de Docker Compose, la construcción con network: host exige un permiso que no se puede pasar a docker compose up. En ese caso basta con construir las imágenes con `docker build --network host` y levantar sin --build, como se explica en el README.

8. Optimización

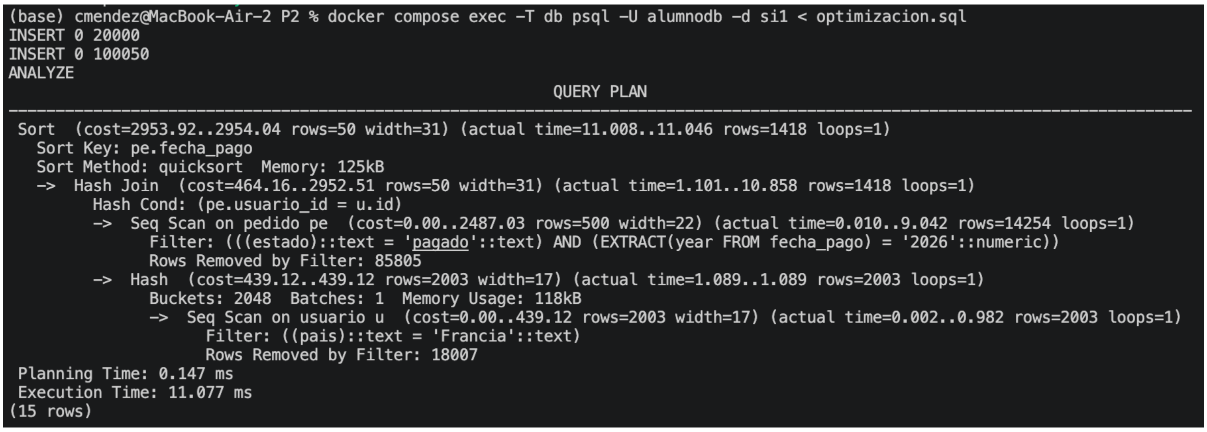
Hemos estudiado la consulta del endpoint /ventas/<año>/<país>, que une Pedido con Usuario y filtra por estado, país y año de la fecha de pago:

```
SELECT pe.id AS pedido_id, u.nombre AS usuario, pe.fecha_pago, pe.total
FROM Pedido pe JOIN Usuario u ON u.id = pe.usuario_id
WHERE pe.estado = 'pagado' AND u.pais = 'Francia'
AND EXTRACT(YEAR FROM pe.fecha_pago) = 2026
ORDER BY pe.fecha_pago;
```

Con los datos de populate.sql cualquier consulta tarda menos de un milisegundo y PostgreSQL siempre recorre las tablas enteras, así que optimizacion.sql empieza generando 20.000 usuarios repartidos entre 10 países y 5 pedidos pagados por usuario (unos 100.000) con fechas aleatorias entre 2020 y 2026, usando generate_series y random(). Tras cada cambio ejecutamos ANALYZE para actualizar las estadísticas y EXPLAIN ANALYZE, que además del plan muestra el tiempo real de ejecución. La consulta devuelve unas 1.400 filas.

Estrategia	**Tiempo**	**Resultado**
Sin optimizar	11,08 ms	Seq Scan en Pedido y Usuario
Índice normal sobre Pedido(fecha_pago)	10,61 ms	No se usa por el EXTRACT
Índice de expresión sobre EXTRACT(YEAR FROM fecha_pago)	3,58 ms	Se usa sin cambiar la API (elegida)

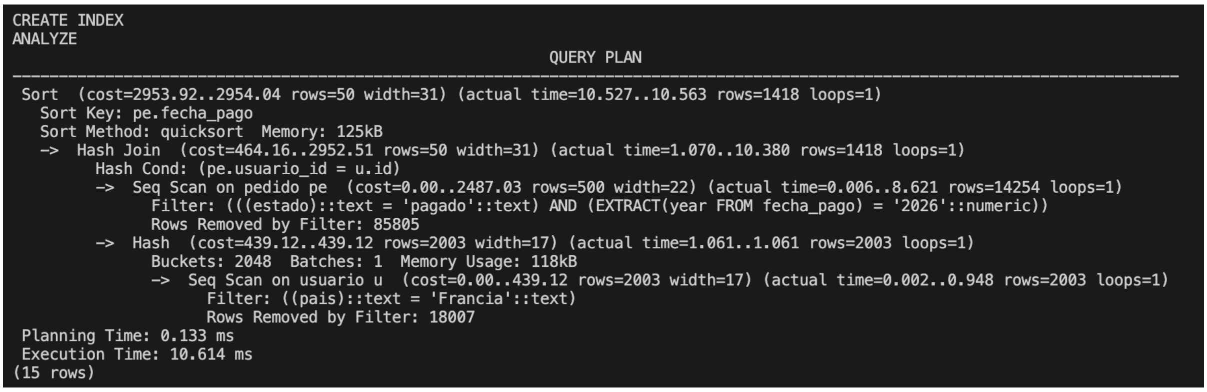
Sin optimizar. El plan recorre secuencialmente los 100.000 pedidos y los 20.000 usuarios y une ambos con un Hash Join. Casi todo el tiempo (unos 9 de los 11 ms) se va en el recorrido de Pedido, donde se descartan unas 86.000 filas. Además, PostgreSQL estima que saldrán 50 filas cuando salen unas 1.400, porque no tiene estadísticas sobre el año de la fecha.



```
(base) cmendez@MacBook-Air-2 P2 % docker compose exec -T db psql -U alumnodb -d si1 < optimizacion.sql
INSERT 0 20000
INSERT 0 100050
ANALYZE
                                                            QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=2953.92..2954.04 rows=50 width=31) (actual time=11.008..11.046 rows=1418 loops=1)
   Sort Key: pe.fecha_pago
   Sort Method: quicksort  Memory: 125kB
   ->  Hash Join  (cost=464.16..2952.51 rows=50 width=31) (actual time=1.101..10.858 rows=1418 loops=1)
         Hash Cond: (pe.usuario_id = u.id)
         ->  Seq Scan on pedido pe  (cost=0.00..2487.03 rows=500 width=22) (actual time=0.010..9.042 rows=14254 loops=1)
               Filter: (((estado)::text = 'pagado'::text) AND (EXTRACT(year FROM fecha_pago) = '2026'::numeric))
               Rows Removed by Filter: 85805
         ->  Hash  (cost=439.12..439.12 rows=2003 width=17) (actual time=1.089..1.089 rows=2003 loops=1)
               Buckets: 2048  Batches: 1  Memory Usage: 118kB
               ->  Seq Scan on usuario u  (cost=0.00..439.12 rows=2003 width=17) (actual time=0.002..0.982 rows=2003 loops=1)
                     Filter: ((pais)::text = 'Francia'::text)
                     Rows Removed by Filter: 18007
 Planning Time: 0.147 ms
 Execution Time: 11.077 ms
(15 rows)
```

Figura 4. Plan sin optimizar

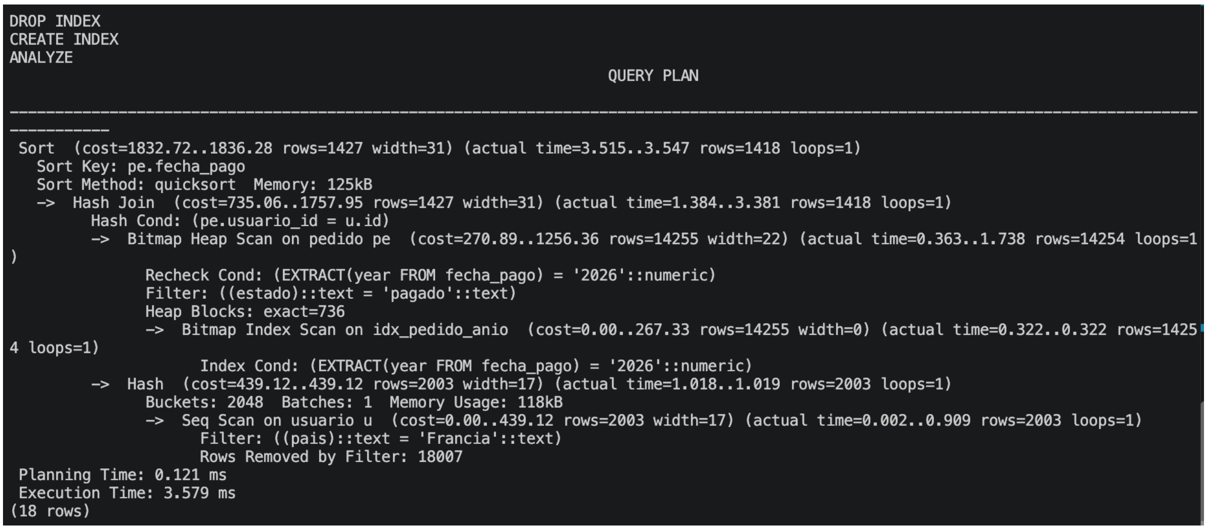
Índice normal sobre fecha_pago. Como el filtro costoso es el de la fecha, lo primero que probamos fue un índice sobre esa columna. El plan no cambia: el índice no se usa en absoluto. La consulta no busca por fecha_pago sino por EXTRACT(YEAR FROM fecha_pago), y el índice está ordenado por fechas completas, no por años, así que para aprovecharlo PostgreSQL tendría que calcular el año de cada fila, que es justo recorrer la tabla. En general, aplicar una función a una columna en el WHERE impide usar el índice normal de esa columna. Como no aporta nada, el script lo elimina.



```
CREATE INDEX
ANALYZE
                                                         QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=2953.92..2954.04 rows=50 width=31) (actual time=10.527..10.563 rows=1418 loops=1)
   Sort Key: pe.fecha_pago
   Sort Method: quicksort  Memory: 125kB
   ->  Hash Join  (cost=464.16..2952.51 rows=50 width=31) (actual time=1.070..10.380 rows=1418 loops=1)
         Hash Cond: (pe.usuario_id = u.id)
         ->  Seq Scan on pedido pe  (cost=0.00..2487.03 rows=500 width=22) (actual time=0.006..8.621 rows=14254 loops=1)
               Filter: (((estado)::text = 'pagado'::text) AND (EXTRACT(year FROM fecha_pago) = '2026'::numeric))
               Rows Removed by Filter: 85805
         ->  Hash  (cost=439.12..439.12 rows=2003 width=17) (actual time=1.061..1.061 rows=2003 loops=1)
               Buckets: 2048  Batches: 1  Memory Usage: 118kB
               ->  Seq Scan on usuario u  (cost=0.00..439.12 rows=2003 width=17) (actual time=0.002..0.948 rows=2003 loops=1)
                     Filter: ((pais)::text = 'Francia'::text)
                     Rows Removed by Filter: 18007
 Planning Time: 0.133 ms
 Execution Time: 10.614 ms
(15 rows)
```

Figura 5. Plan con el índice normal sobre fecha_pago: sigue haciendo Seq Scan

Índice de expresión**.** Un índice de expresión guarda ya calculado el resultado de EXTRACT(YEAR FROM fecha_pago), que es exactamente lo que busca la consulta, así que PostgreSQL lo usa (Bitmap Index Scan) sin que haya que cambiar nada en la API. El tiempo baja de unos 11 ms a unos 3,6 ms, unas tres veces más rápido. Además, al tener estadísticas de esa expresión, PostgreSQL pasa a estimar correctamente el número de filas (1.427 estimadas frente a 1.418 reales), lo que le permite elegir mejor el plan.



```
DROP INDEX
CREATE INDEX
ANALYZE
                                                                 QUERY PLAN

-------------------------------------------------------------------------------------------------------------------------------------
-----------
 Sort  (cost=1832.72..1836.28 rows=1427 width=31) (actual time=3.515..3.547 rows=1418 loops=1)
   Sort Key: pe.fecha_pago
   Sort Method: quicksort  Memory: 125kB
   ->  Hash Join  (cost=735.06..1757.95 rows=1427 width=31) (actual time=1.384..3.381 rows=1418 loops=1)
         Hash Cond: (pe.usuario_id = u.id)
         ->  Bitmap Heap Scan on pedido pe  (cost=270.89..1256.36 rows=14255 width=22) (actual time=0.363..1.738 rows=14254 loops=1
)
               Recheck Cond: (EXTRACT(year FROM fecha_pago) = '2026'::numeric)
               Filter: ((estado)::text = 'pagado'::text)
               Heap Blocks: exact=736
               ->  Bitmap Index Scan on idx_pedido_anio  (cost=0.00..267.33 rows=14255 width=0) (actual time=0.322..0.322 rows=1425
4 loops=1)
                     Index Cond: (EXTRACT(year FROM fecha_pago) = '2026'::numeric)
         ->  Hash  (cost=439.12..439.12 rows=2003 width=17) (actual time=1.018..1.019 rows=2003 loops=1)
               Buckets: 2048  Batches: 1  Memory Usage: 118kB
               ->  Seq Scan on usuario u  (cost=0.00..439.12 rows=2003 width=17) (actual time=0.002..0.909 rows=2003 loops=1)
                     Filter: ((pais)::text = 'Francia'::text)
                     Rows Removed by Filter: 18007
 Planning Time: 0.121 ms
 Execution Time: 3.579 ms
(18 rows)
```

Figura 6. Plan con el índice de expresión idx_pedido_anio

Otra opción habría sido reescribir la condición como un rango de fechas (fecha_pago >= '2026-01-01' AND fecha_pago < '2027-01-01'), con la que el índice normal sí se usaría, pero obliga a cambiar la consulta de la API, mientras que el índice de expresión funciona con la consulta tal y como está. Los tiempos varían ligeramente en cada ejecución porque los datos son aleatorios, pero las proporciones se mantienen.

9. Limitaciones

Somos conscientes de algunas limitaciones. Las contraseñas se guardan con SHA-256 sin sal, heredado de la práctica 1; en un caso real se usaría un algoritmo como bcrypt. El token no caduca por tiempo (solo con el logout) y depende de un secreto compartido guardado en el proyecto; en un caso real se usaría JWT con expiración. Cualquier usuario autenticado puede crear y borrar películas, porque el enunciado no pide roles de administrador. El stock se reserva al añadir al carrito, por lo que un carrito abandonado retiene unidades hasta que se vacía. Por último, al borrar un usuario sus compras desaparecen también del historial de ventas.